

Apache Kafka와 Pub/Sub 패턴 살펴보기

박연종

목차

RabbitMQ

많은 기업에서 사용하고 있는 Kafka

Kafka에서 사용하는 용어와 메커니즘

Kafka, 어떻게 활용하고 있을까?

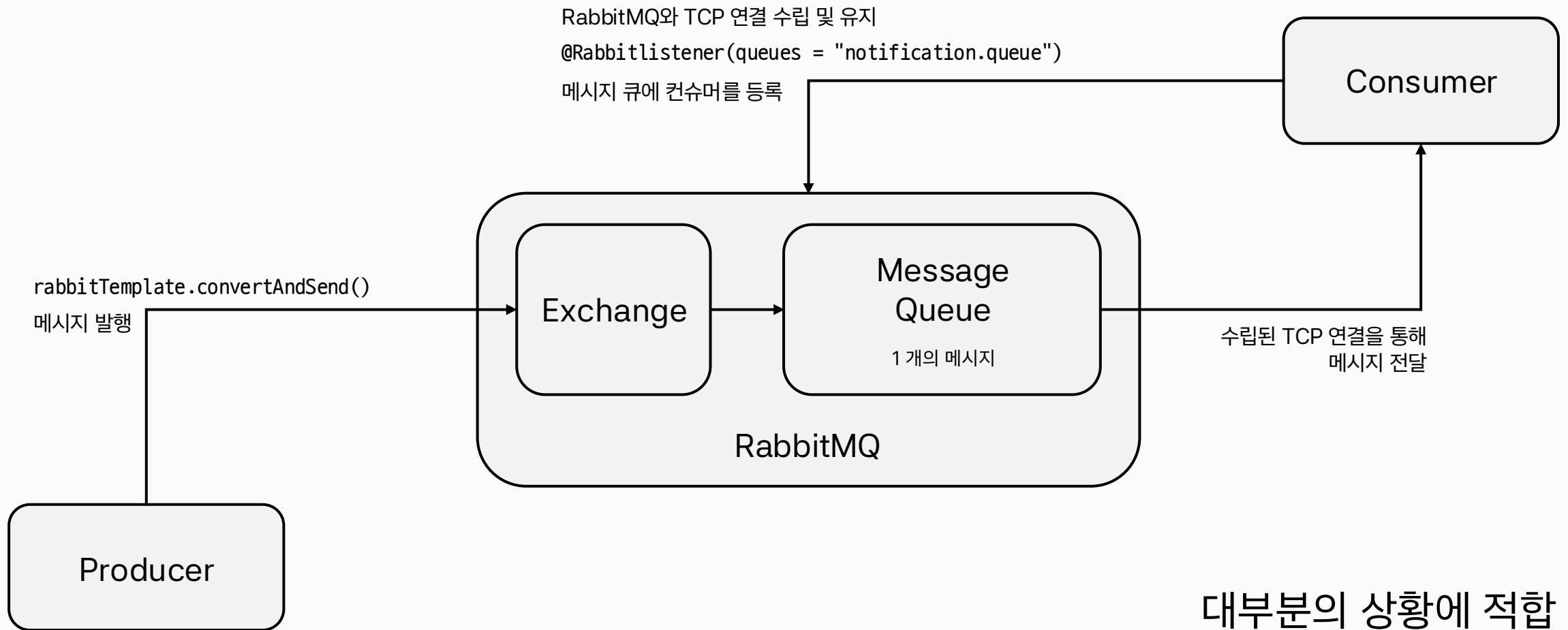
RabbitMQ

많은 기업에서 사용하고 있는 Kafka

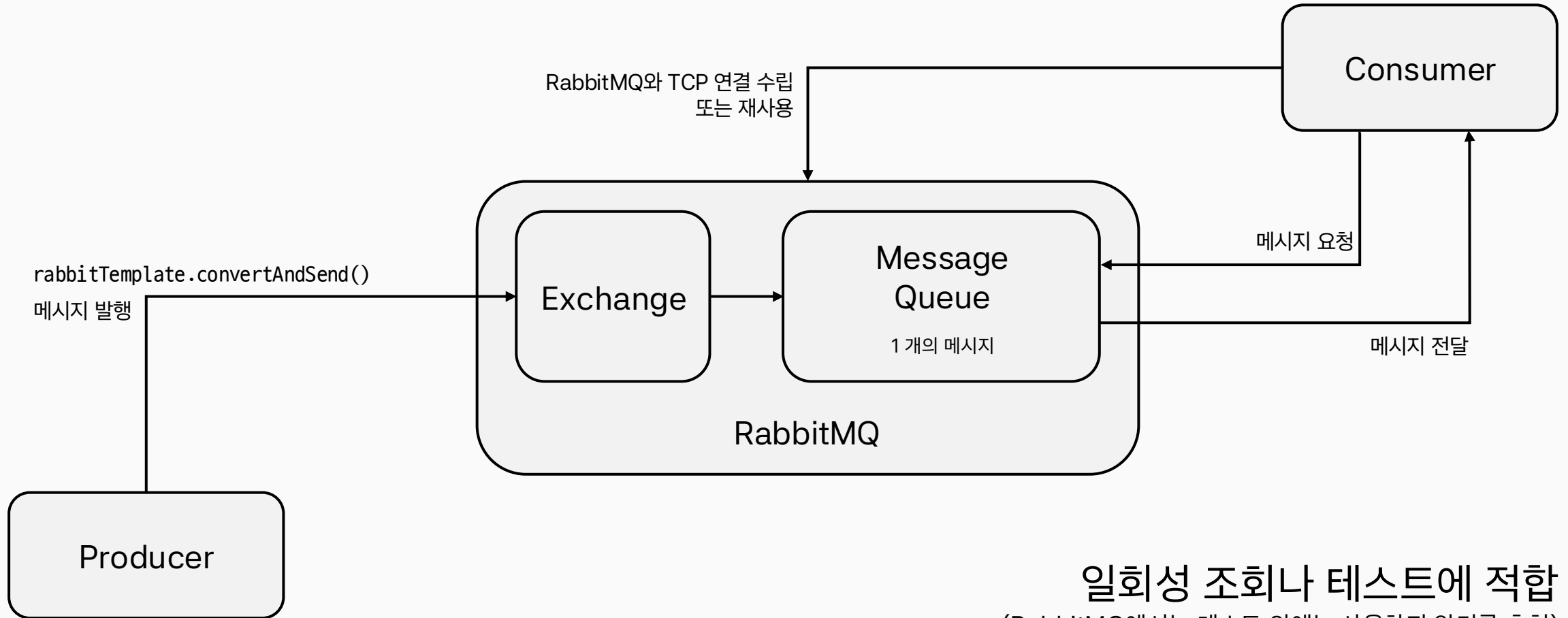
Kafka에서 사용하는 용어와 메커니즘

Kafka, 어떻게 활용하고 있을까?

RabbitMQ는 메시지를 어떻게 가져올까? (basic.consume)



RabbitMQ는 메시지를 어떻게 가져올까? (basic.get)



일회성 조회나 테스트에 적합
(RabbitMQ에서는 테스트 외에는 사용하지 않기를 추천)

RabbitMQ는 여러 메시지 큐에서 잘 선택해 메시지를 전달할까?

Direct – 정확히 일치하는 큐에 전달

로그 레벨에 따라 로그를 서로 다른 큐로 전달

이메일, 문자 메시지, 푸시 등 알림 종류에 따라 큐로 전달

Topic – 패턴에 일치하는 모든 큐에 전달

로그 발생 도메인과 로그 레벨을 조합해 필요한 로그만 전달

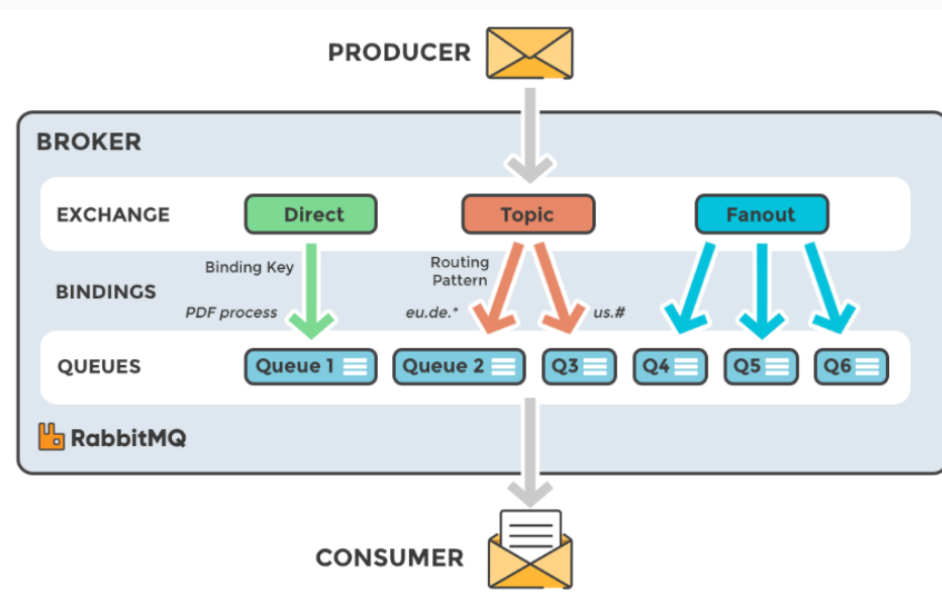
서비스, 이벤트 종류, 지역을 조합해 필요한 이벤트만 전달

Fanout – 바인딩된 모든 큐에 전달

캐시 무효화, 설정 변경 등 이벤트를 동시에 전달

Headers – 헤더 조건에 일치하는 모든 큐에 전달

지역, 파일 형식, 우선 순위 등 여러 속성 조합에 따라 큐로 전달



RabbitMQ

많은 기업에서 사용하고 있는 Kafka

Kafka에서 사용하는 용어와 메커니즘

Kafka, 어떻게 활용하고 있을까?

Apache Kafka, 많은 기업들이 채택하는 기술 스택

Intelligence Targeting에서 사용하는 기술이에요

- Go, gRPC, gRPC-Gateway
- DynamoDB, Redis, MySQL, BigQuery, Kafka
- Datadog, Sentry

이와 더불어, 팀에서는 Claude Code와 Codex 같은 AI 코딩 도구를 개발 전반에 활용해요

당근 인턴 채용 (Backend)

이런 분이면 더 좋아요!

- 대규모 트래픽 환경에서 분산 시스템 또는 마이크로서비스를 설계·운영해 본 분
- 피드, 추천, 개인화 등 콘텐츠 서빙 도메인을 경험해 본 분
- A/B 테스트 등 데이터 기반 실험을 설계하고 검증해 본 분
- 직군을 넘나들며 프로젝트를 처음부터 끝까지 만들어 본 분 (사이드 프로젝트도 좋아요)
- Go, Kafka, Kubernetes 등 팀의 기술 스택을 깊게 사용해 본 분

당근 정규직 채용 (Backend)

합류하면 함께 할 업무예요

- Flink와 Kafka를 기반으로 서비스에 필요한 실시간 데이터 파이프라인을 설계하고 구축해요.
 - 기존 데이터 인프라를 기반으로 서비스에 필요한 스트리밍 파이프라인의 구조와 운영 기준을 직접 설계하고 만들어갈 수 있어요.
 - 서비스에서 발생한 데이터를 필요한 형태로 가공하고 다른 서비스에서 안정적으로 활용할 수 있도록 전달해요.
 - 데이터의 생성부터 적재·배포·모니터링·변경까지 전체 과정을 안정적으로 운영해요.

토스뱅크 정규직 채용 (Data Platform Engineer)

Responsibilities

- Client, Back-End 전반에서 보안 요구사항을 정의하고 구현합니다.
- API gateway, MCP 서버/클라이언트, Provider 연동 구간에서 보안위험을 식별하고 완화합니다.
- Next.js 기반 운영 콘솔에서 UX를 개선합니다.
- OTLP trace, anomaly event, Datadog, OpenTelemetry, logs, metrics를 활용해 보안 팀 response 체계를 구축합니다.
- Redis, PostgreSQL, MongoDB, Kafka 등 플랫폼 의존성의 설정, 데이터 보존, 감사, 장애 시 다.
- Kubernetes, Helm, Docker, GitLab CI/CD 환경에서 서비스를 개발하고 운영합니다.
- SRE, 인프라, Back-End, Front-End, 보안 조직과 협업해 보안 요구사항을 실현 가능한 플랫폼 기능으로 구체화합니다.

42dot 정규직 채용 (Security Engineer)

[개발환경]

아래는 현재 사용 중인 주요 기술이며, 해결하려는 문제와 운영 환경에 따라 적절한 기술은 다를 수 있습니다.

- Language / Backend: Python, FastAPI
- Data / Streaming: Spark, Flink, Kafka, Redis, Airflow
- AI / Search: LLM, RAG, VectorDB, Elasticsearch
- Infrastructure: AWS, Kubernetes
- Evaluation / Monitoring: Langfuse, MLflow, Prometheus, Grafana
- Collaboration / CI/CD: GitLab, Jira, Confluence, Slack, GitLab CI, ArgoCD

우아한형제들 정규직 채용 (데이터엔지니어링)

직무소개

CJ올리브영의 온·오프라인 옴니채널 비즈니스에서 발생하는 방대한 데이터를 트랜잭션 가치를 창출하는 데이터 플랫폼을 구축합니다.

단순한 데이터 통합을 넘어, Kafka 기반 실시간 스트리밍과 대용량 배치 처리가 결합된 하이브리드 환경을 운영하며,

Apache Iceberg 중심의 레이크하우스(Lakehouse) 아키텍처를 통해 데이터 거버넌스와 성능을 동시에 확보하는 역할을 수행합니다.

CJ올리브영 정규직 채용 (Data Engineer)

리디에서 Kafka를 사용하는 법

내부 API 활용에서 Kafka 기반으로

RIDI

LINE에서 Kafka를 사용하는 방법 - 1편

 Kawamura Yuto 2019-10-28

LINE에서 엔지니어로 재직하고 있습니다. Apache Kafka, Apache HBase 같은 OSS 기술을 바탕으로 LINE의 핵심 데이터 파이프라인과 스토리지를 개발, 관리합니다.

LY Corporation

메시지 대량 발송을 위한 Kafka와 Bulk Insert 적용기 (Feat. Braze)

 yumi Lee

Follow

19 min read · Jul 17, 2023

펫프렌즈

Apache Kafka

체코의 소설가, 프란츠 카프카의 이름에서 유래한
오픈소스 분산 이벤트 스트리밍 플랫폼

개발자 Jay Kreps(Confluent 공동 창업자)에 따르면,

- 쓰기에 최적화된 시스템
- 자신이 프란츠 카프카를 좋아함
- 이름이 멋있게 들림

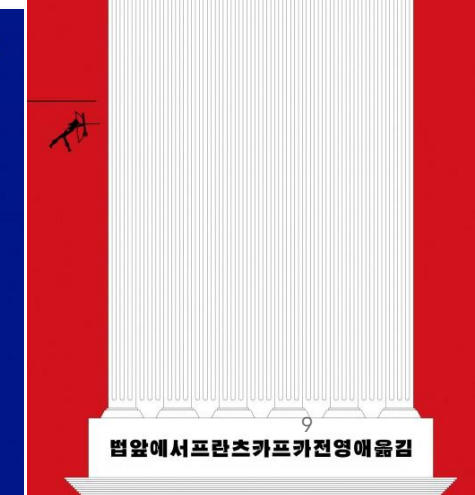
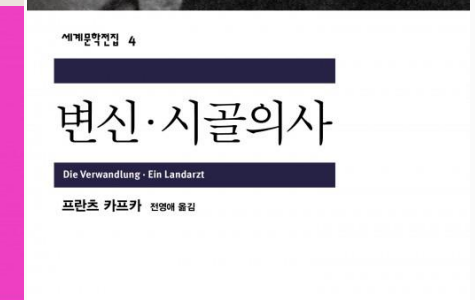
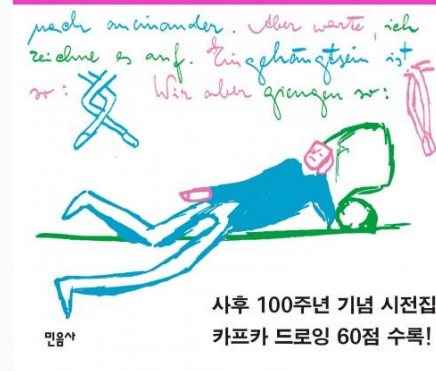
이와 같은 이유로 Kafka라는 이름을 선택

LinkedIn에서 개발되어
Apache Software Foundation 프로젝트로 발전

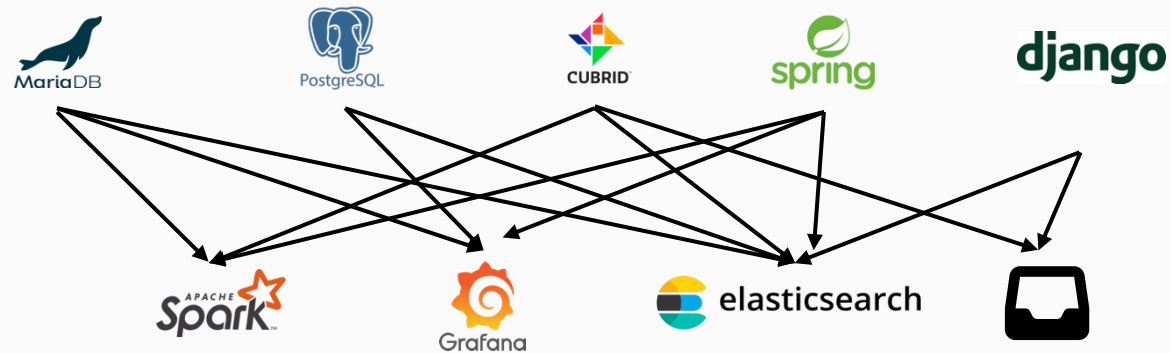
분산 이벤트 스트리밍 플랫폼?

여러 시스템에서 실시간으로 계속 발생하는 데이터를 분산 저장하고,
여러 시스템이 필요할 때 데이터를 읽을 수 있도록 하는 시스템

그냥, 대규모 데이터를 처리할 수 있는 메시지 큐.



Apache Kafka 도입 이전의 환경



엔드 투 엔드 방식의 구조

데이터를 보내는 시스템과 데이터를 사용하는 시스템을 각각 직접 연결해야 함

데이터 연동 복잡성 증가

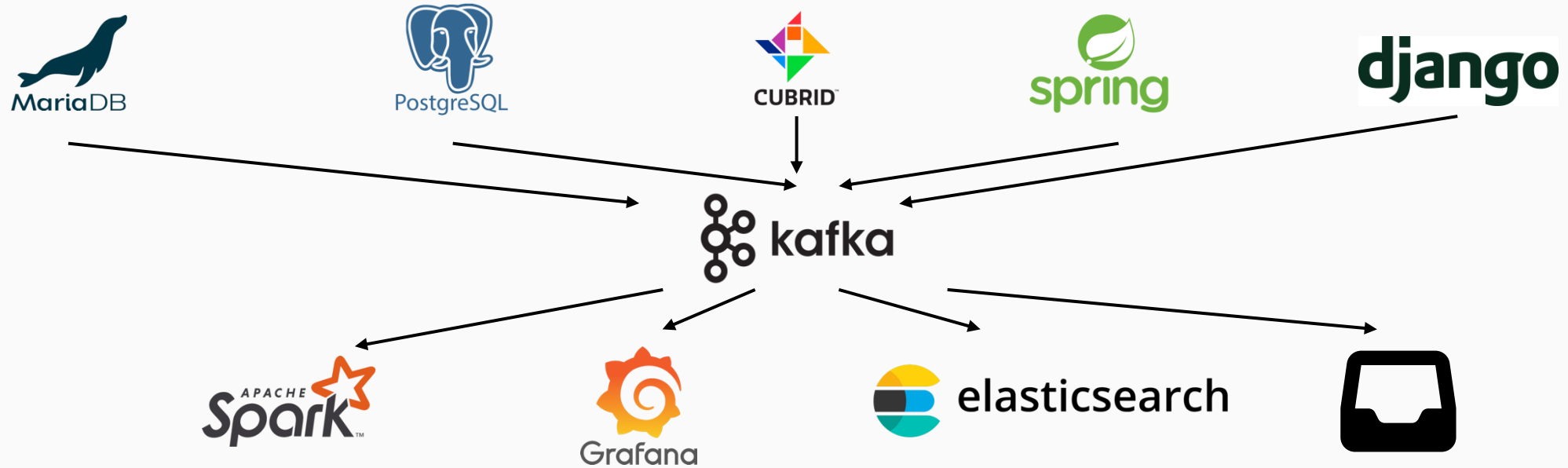
연결마다 하드웨어, 운영체제, 데이터 형식, 전송 방식, 장애 처리 등을 고려해 구현

서비스에서 다양한 로그와 데이터가 지속 발생

데이터 급증 시 각 연결마다 처리량 증가를 고려해야 해 확장에 엄청난 노력이 필요

여러 시스템에 데이터를
안정적으로 전달하면서도,
데이터 급증 시에도
확장이 용이한 시스템이 필요

Apache Kafka 도입 이후의 환경



데이터 생성(Producer)과 데이터 사용 시스템(Consumer)이 서로를 알 필요가 없어짐
하나의 데이터를 여러 Consumer가 독립적으로 활용 가능
대량 이벤트를 빠르게 저장하고, 전달할 수 있도록 설계
수평 확장이 용이해 데이터 급증 시 데이터 처리 능력을 향상
다양한 데이터 연동 및 처리 기능 제공

RabbitMQ

많은 기업에서 사용하고 있는 Kafka

Kafka에서 사용하는 용어와 메커니즘

Kafka, 어떻게 활용하고 있을까?

Producer, Consumer, Consumer Group, Topic

Producer

Kafka의 Topic에 이벤트를 발행하는 애플리케이션

Consumer

Kafka의 Topic을 구독해 이벤트를 읽는 애플리케이션

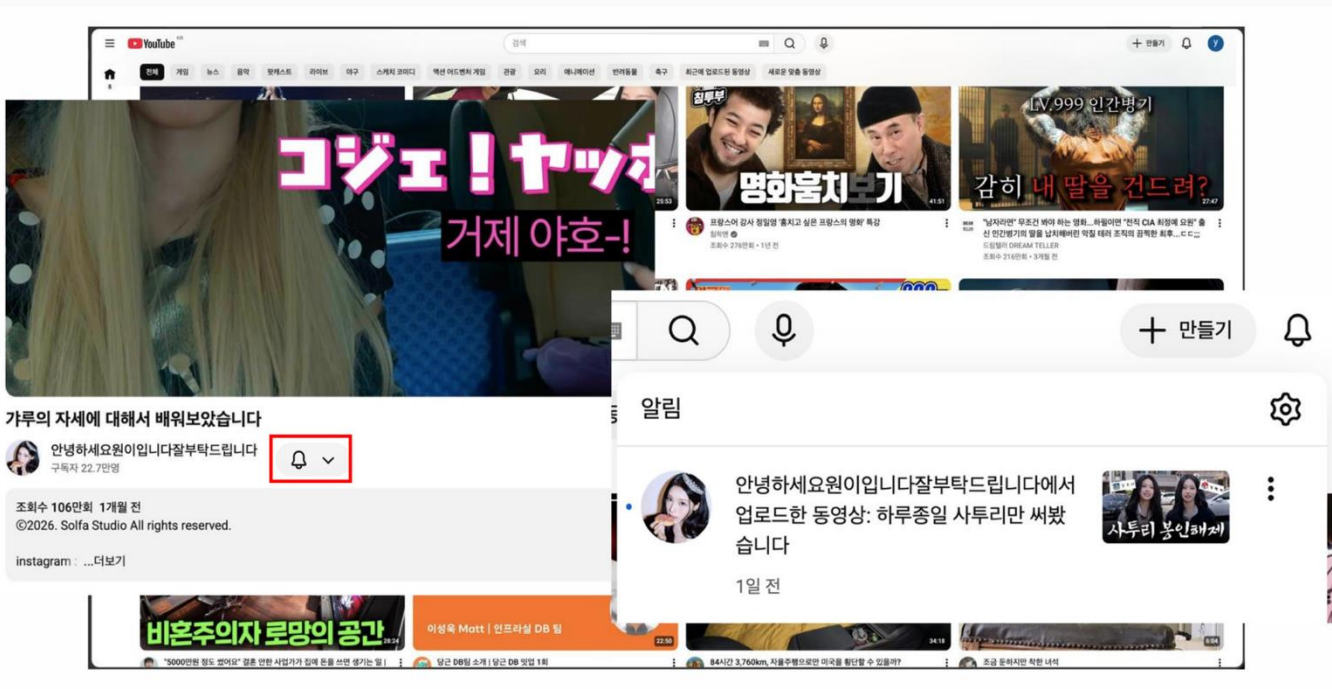
Consumer Group

같은 Topic을 구독하는 Consumer의 논리적 모임

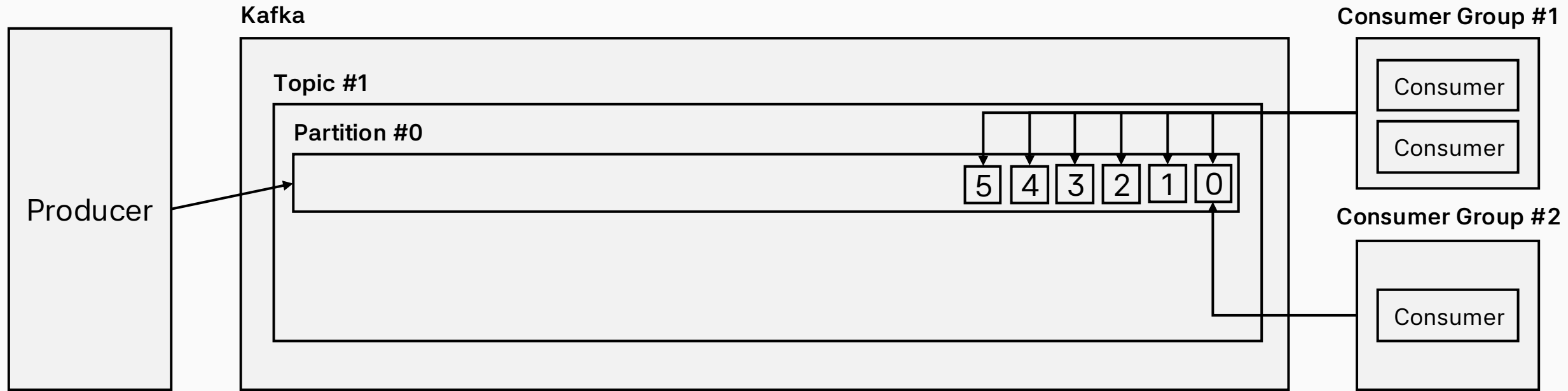
Topic

이벤트가 저장되는 논리적 공간

메시지를 구분하는 단위



Record, Partition, Offset

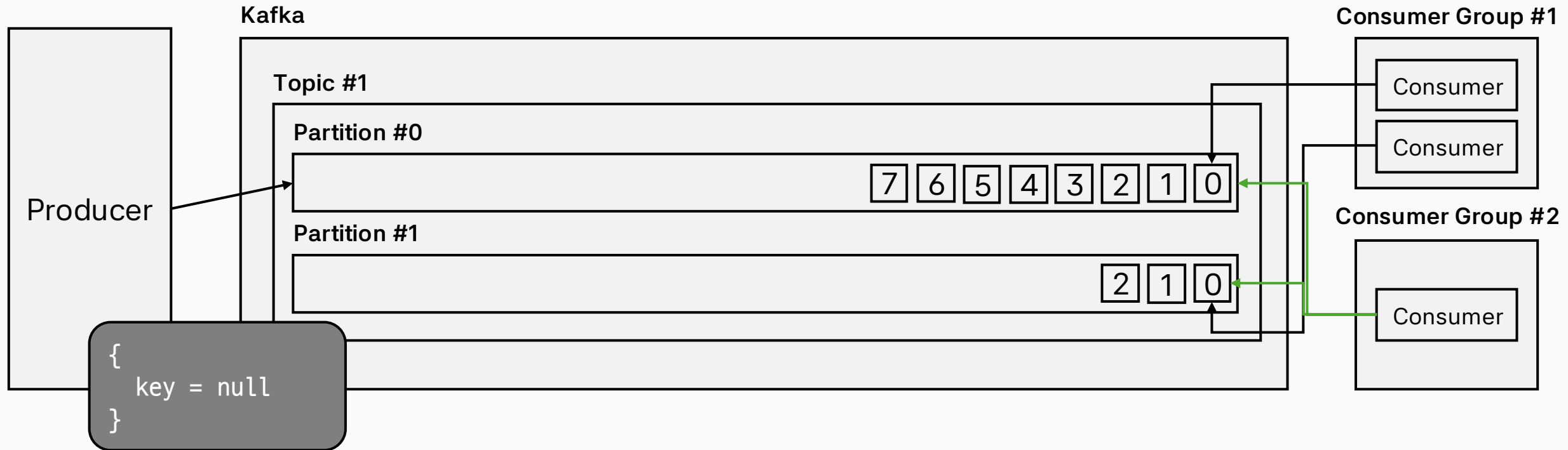


Record 타임스탬프, 오프셋, 헤더, 키, 메시지 값 등이 저장되는 데이터 단위

Partition 레코드를 저장하는 물리적인 파일
하나의 토픽은 하나 이상의 파티션으로 구성
데이터 처리 분산을 위해 파티션을 증가할 수 있으나, 줄일 수는 없으므로 신중하게 늘려야 함

Offset 특정 파티션 내 레코드를 식별하는 값

Record, Partition, Offset



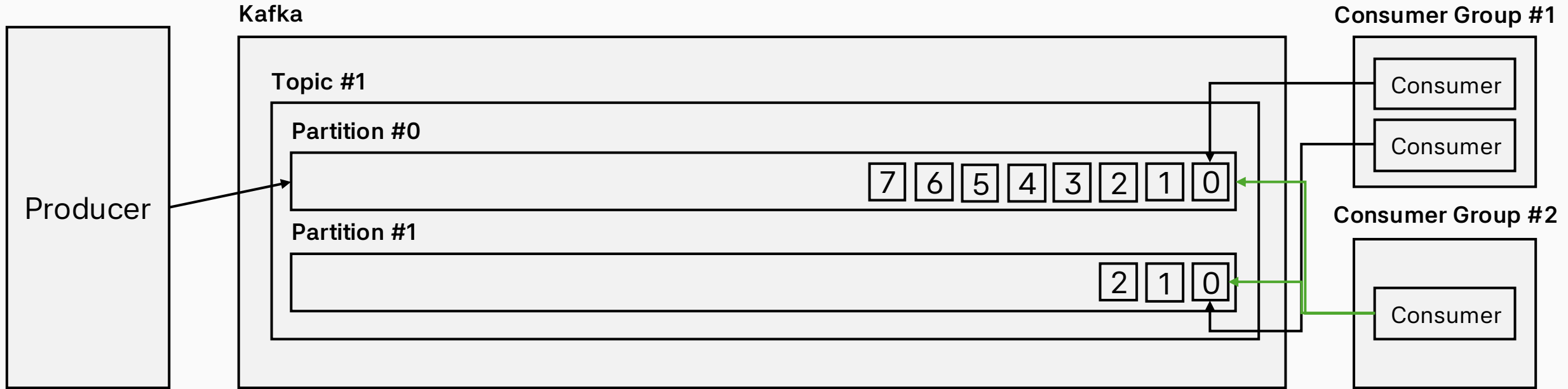
Producer가 지정한 키가 null이면 파티션을 라운드 로빈으로 할당

```
{  
  key = "kitssl"  
}
```

$\text{Positive}(\text{MurmurHash2}(\text{key})) \% \text{partitionCount}$

Producer가 키를 지정하면 키의 해시 값을 구해 특정 파티션에 할당

Record, Partition, Offset



Partition 내부만 순서가 보장되므로 순서가 중요한 경우 같은 Partition으로 보내야 함

Producer가 생성한 메시지를 Consumer가 처리해 보자

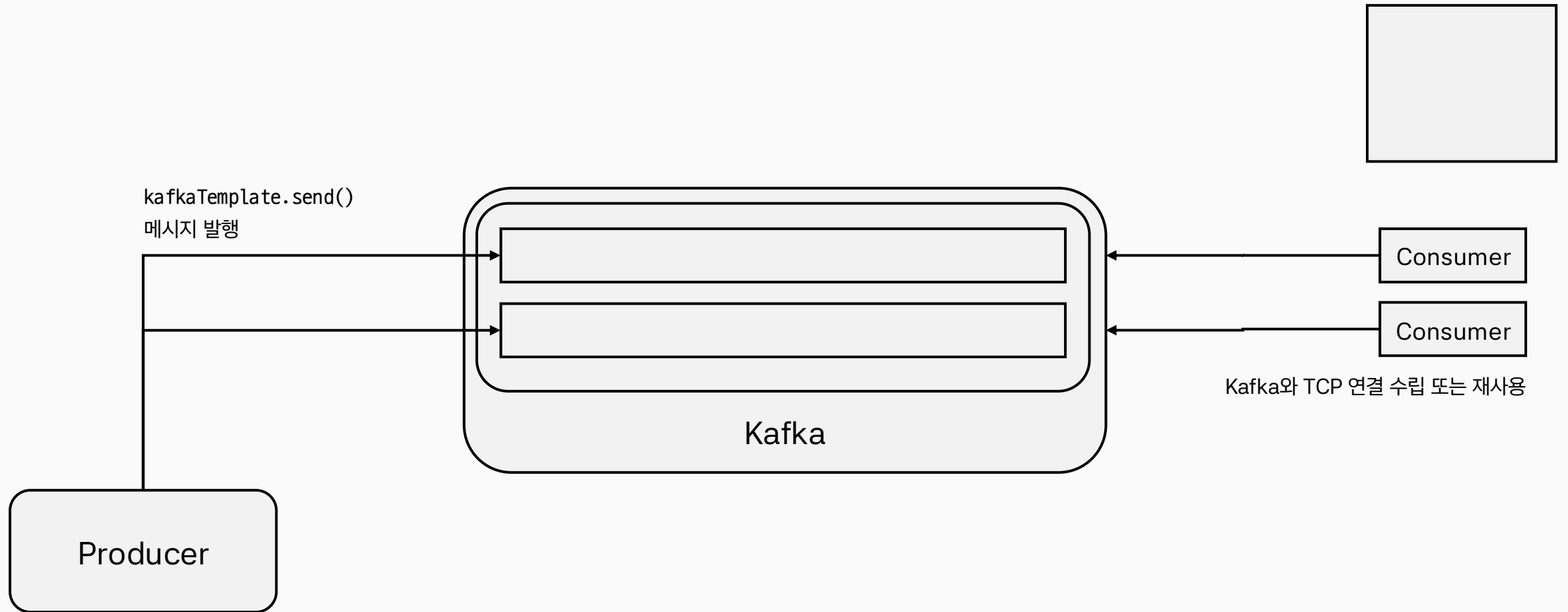
Producer

```
T14:13:11.716+09:00 INFO 12512 --- [producer-app] [-app-producer-1] c.e.kafkademoproducer.VideoController : Video uploaded topic=demo-topic partition=0 offset=16 eventId=e8ba0d15-b99d-48b1-bfc3-7638cebdb3a channelId=SBS 뉴스 videoId=u6HFW7qcHBQ title=결국 우려했던 일 터졌다..국민 상어 '부캤이' 상태가 / SBS 8뉴스
T14:15:35.664+09:00 INFO 12512 --- [producer-app] [-app-producer-1] c.e.kafkademoproducer.VideoController : Video uploaded topic=demo-topic partition=0 offset=17 eventId=54c10c80-063e-4a94-9365-07f8d19ba0cc channelId=PCMag videoId=3cS86RBG0Iw title=Windows 이벤트에서 공개된 Surface Laptop Ultra 보기
```

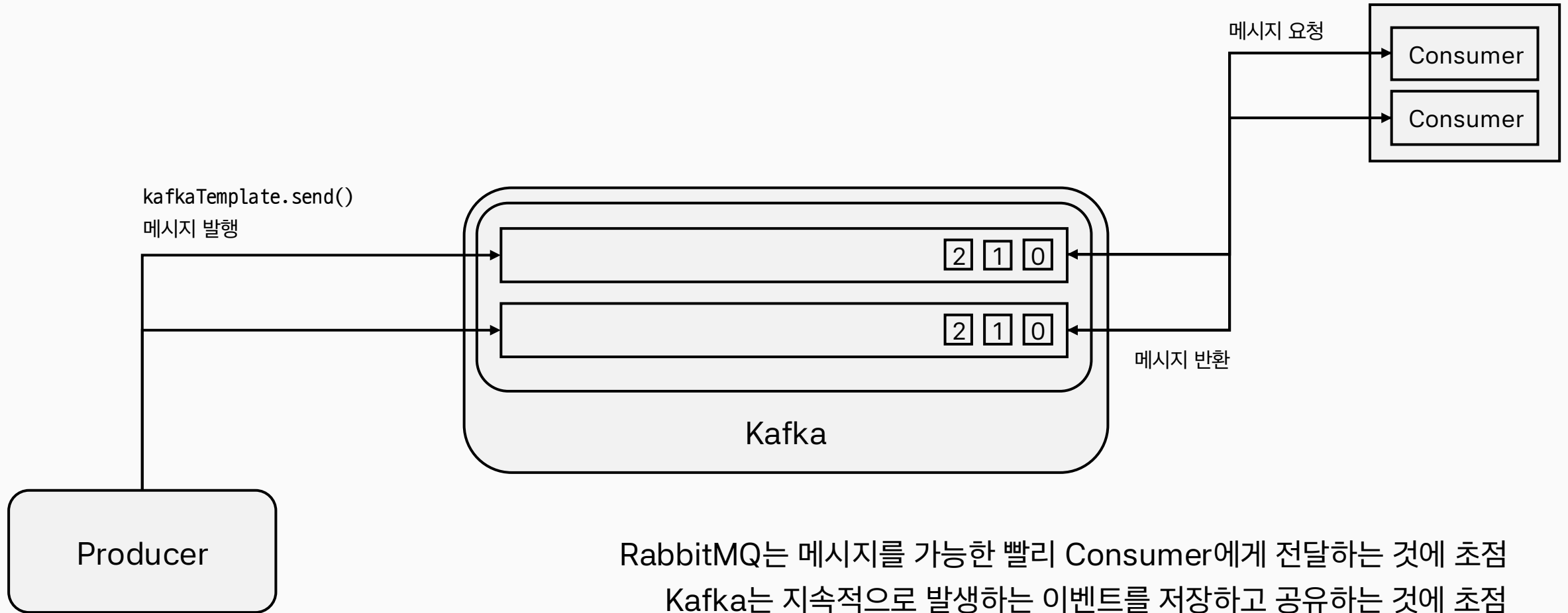
Consumer - notification

```
T14:13:11.843+09:00 INFO 16128 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Consumed video role=notifications topic=demo-topic partition=0 offset=16 eventId=e8ba0d15-b99d-48b1-bfc3-7638cebdb3a channelId=SBS 뉴스 videoId=u6HFW7qcHBQ
T14:13:11.845+09:00 INFO 16128 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.NotificationHandler : Notification summary eventId=e8ba0d15-b99d-48b1-bfc3-7638cebdb3a channelId=SBS 뉴스 recipients=[] recipientCount=0
T14:13:11.846+09:00 INFO 16128 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Processing succeeded role=notifications topic=demo-topic partition=0 offset=16
T14:13:11.975+09:00 INFO 16128 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Offset committed role=notifications topic=demo-topic partition=0 committed Offset=17
T14:15:35.664+09:00 INFO 16128 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Consumed video role=notifications topic=demo-topic partition=0 offset=17 eventId=54c10c80-063e-4a94-9365-07f8d19ba0cc channelId=PCMag videoId=3cS86RBG0Iw
T14:15:35.671+09:00 INFO 16128 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.NotificationHandler : Notification summary eventId=54c10c80-063e-4a94-9365-07f8d19ba0cc channelId=PCMag recipients=[] recipientCount=0
T14:15:35.674+09:00 INFO 16128 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Processing succeeded role=notifications topic=demo-topic partition=0 offset=17
T14:15:35.687+09:00 INFO 16128 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Offset committed role=notifications topic=demo-topic partition=0 committed Offset=18
```

Kafka는 메시지를 어떻게 가져올까?



Kafka는 메시지를 어떻게 가져올까?



Offset은 어떻게 관리할까?

파티션은 추가만 가능(append-only)한 파일

파티션별로 Offset은 계속 증가하며, 64비트 정수형. (초당 10억 개의 메시지가 와도 오버플로우까지 약 292년 소요)

Consumer는 다음에 읽을 Offset을 커밋해 Kafka에 저장할 수 있음

Kafka 내부 Topic인 `__consumer_offsets`에 저장됨

커밋된 Offset이 없으면 맨 처음 Offset을 사용하거나, 가장 마지막 Offset을 사용하도록 설정할 수 있음

커밋은 자동으로 수행되나, 리밸런싱 후 데이터 중복 또는 유실 문제가 발생할 우려가 있어 이를 허용하지 않는 시스템은 대체로 설정을 끄

동기 커밋 레코드 처리 순서를 보장하며 커밋 완료 시까지 블록

비동기 커밋 레코드 처리 순서를 보장하지 않으며 커밋 실패 시 재시도로 인해 최신 Offset이 과거 Offset으로 덮어쓰워질 수 있음

Partition의 Record는 언제 삭제될까?

Kafka는 파티션을 여러 세그먼트 파일로 나누어 관리

기본적으로 모든 레코드가 7일(604800000ms) 경과한 세그먼트 파일을 삭제
삭제되어도 Offset은 당겨지지 않고, 계속 증가함

청소 정책을 delete에서 compact로 설정하면 최신 Key만 남김
delete와 compact를 함께 적용할 수도 있음

Broker, Cluster, Replication, ISR

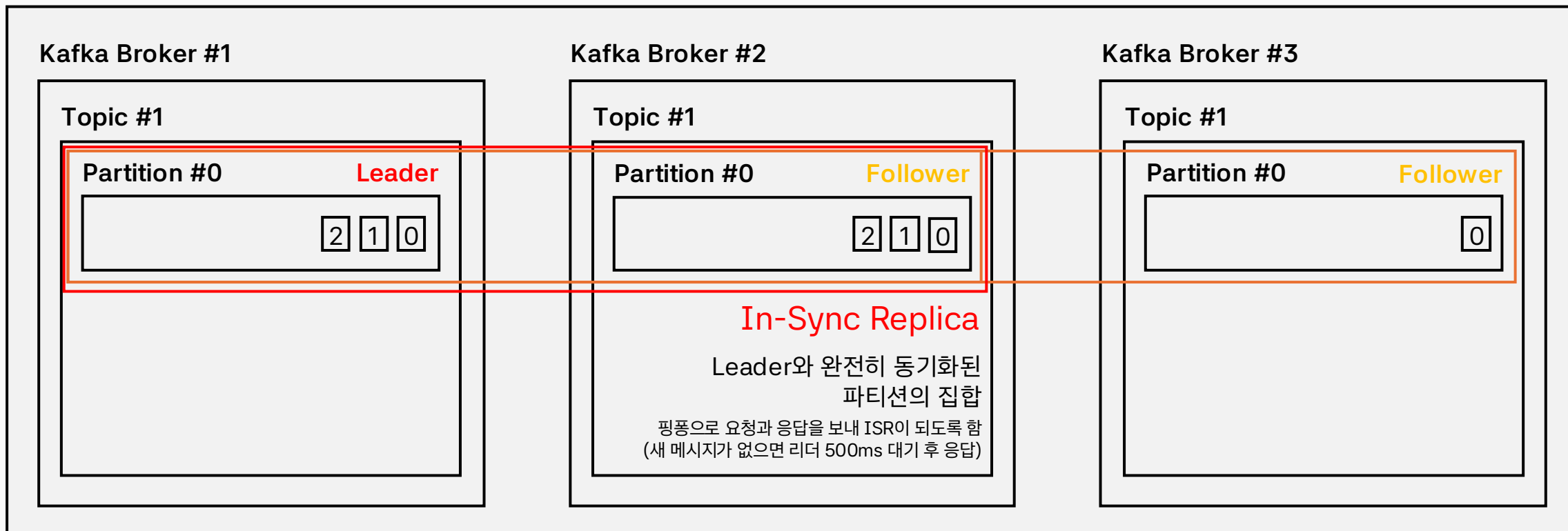
Kafka Broker Kafka가 구성된 서버의 단위

Kafka Cluster 여러 대의 Broker로 구성된 Kafka 시스템

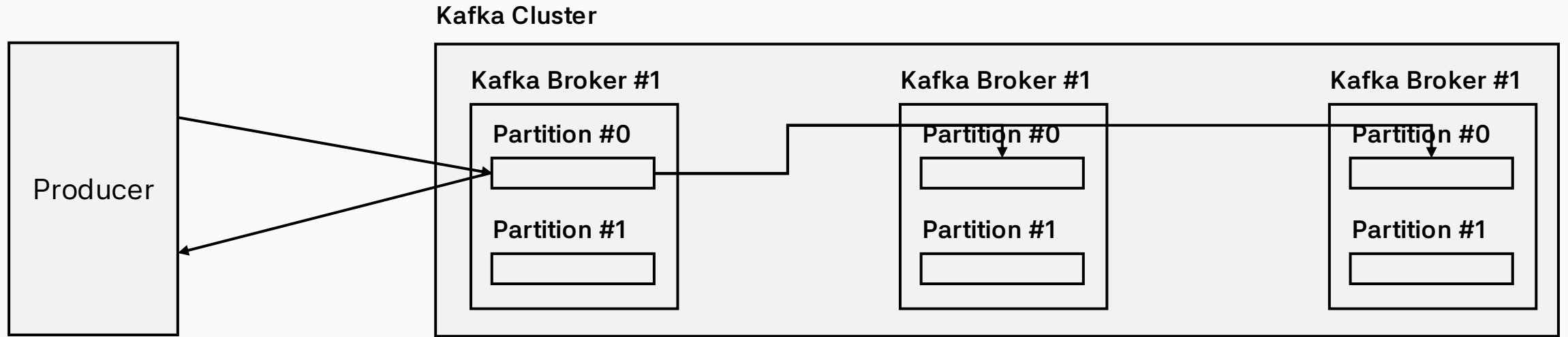
Replication Factor는 브로커 개수를 초과할 수 없음

--partitions 1
--replication-factor 3

Kafka Cluster



ACK



ack = 0 응답을 보내지 않으며, 전송을 보장하지 않음

ack = 1 Leader 파티션에 저장되면 응답을 보냄
Leader 파티션 장애 시 메시지가 유실될 수 있음

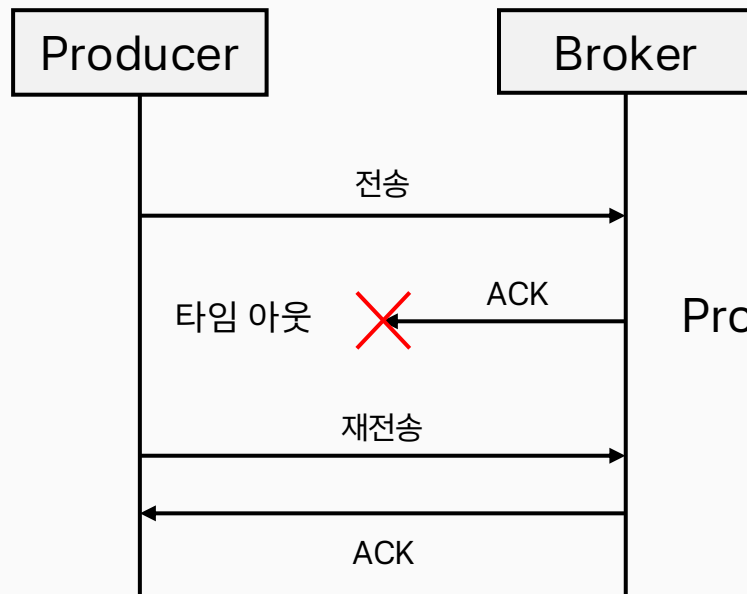
ack = all 지정한 최소 개수만큼 Replica에 저장되면 응답을 보냄

ack = -1 전송을 보장하나 속도가 현저히 느림

Producer가 메시지 전송에 실패하면?

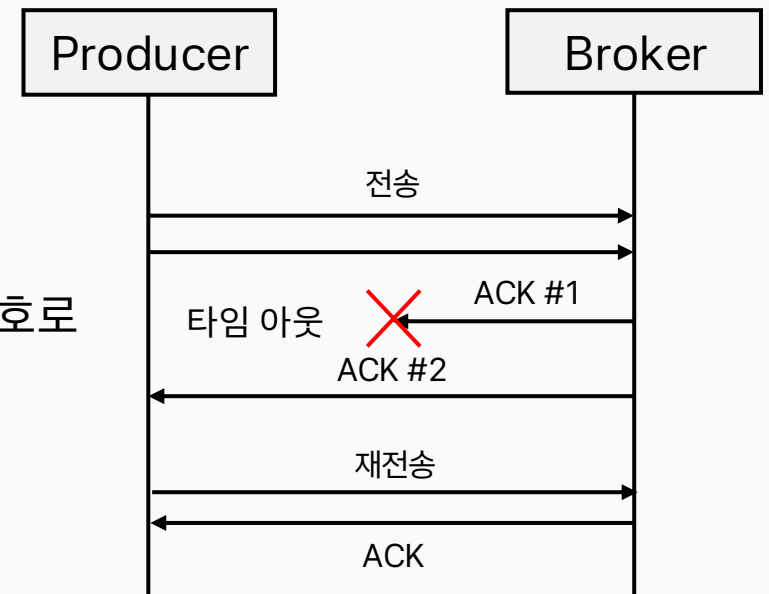
- 네트워크 오류 등에 의한 전송 타임 아웃
- 리더 파티션 장애로 인한 새로운 리더 파티션 선출
- 메시지 크기 초과
- 프로듀서의 버퍼 크기 초과
- ...
- 재시도
- 기록 후 작업(수동/자동)

재시도 시 주의사항



ACK 유실 시 중복 전송 가능

역등성 설정을 켜면(기본)
Producer ID와 Producer가 지정한 순서 번호로
이미 처리한 요청인지 구분하거나
순서를 검사해 순서 변경을 방지할 수 있음



여러 요청 전송 시 순서 변경 가능

Consumer가 메시지 처리 중 오류가 발생하면?

재시도 및 주의사항

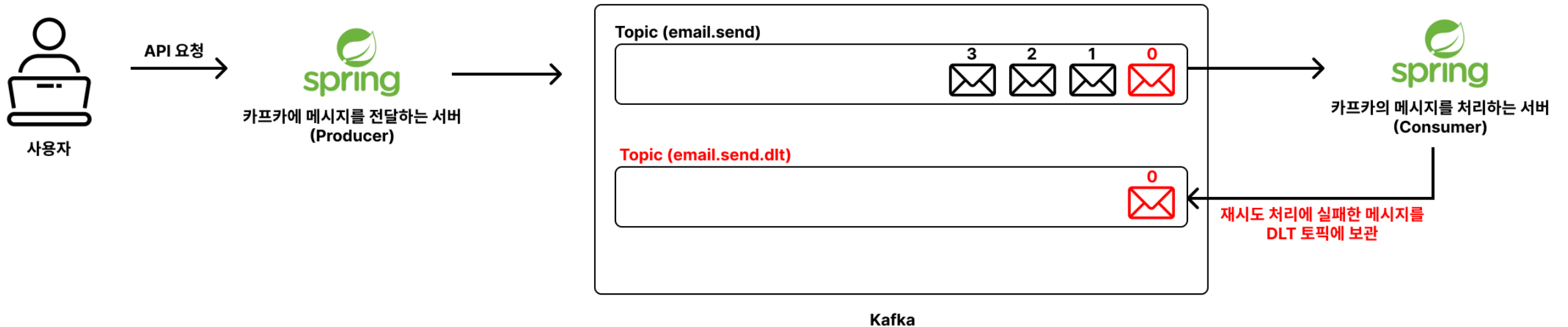
Consumer는 동일한 메시지가 다시 처리될 수 있음에 유의해야 함

데이터 특성에 따라 타임스탬프, 고유 식별자 등을 통해 중복 처리를 방지해야 함

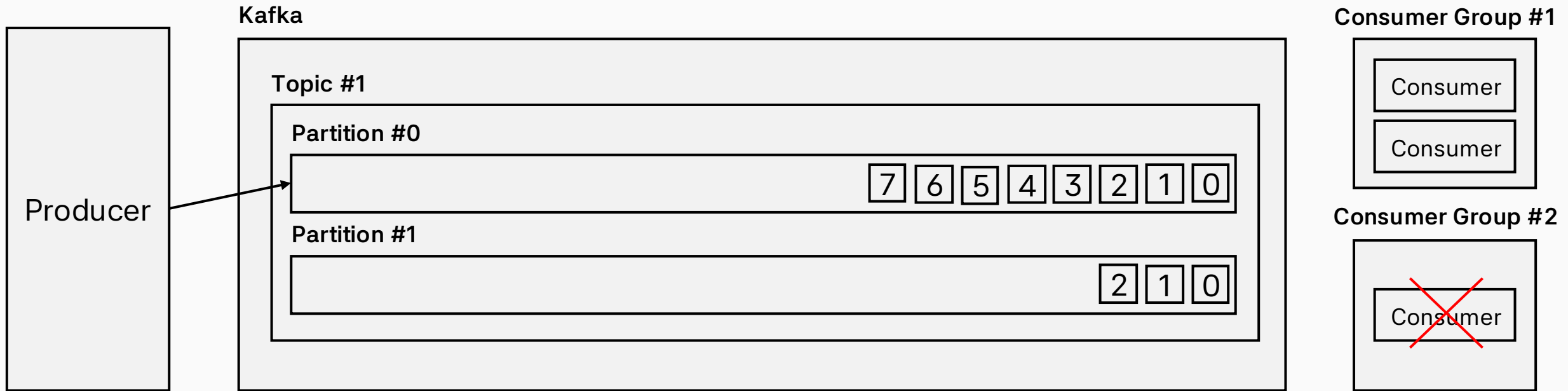
처리한 메시지를 데이터베이스에 기록해 검사, 마지막 처리 시각과 비교하는 등의 방법이 있음

DLT(Dead Letter Topic) 기록 후 작업(자동/수동)

DLT는 기본 생성이 아닌, Spring Kafka가 지원하므로 수동으로 생성함



Consumer가 메시지 처리 중 오류가 발생하면?



Consumer Group #2에서 하나뿐인 Consumer에 장애 발생 시 Consumer 프로세스/인스턴스를 재시작해야 함
Consumer 복구 시 마지막으로 커밋된 오프셋부터 처리를 계속

Consumer 장애 복구 시 마지막 커밋된 오프셋부터 처리를 시작한다

Producer

```
T14:19:06.646+09:00 INFO 12512 --- [producer-app] [-app-producer-1] c.e.kafkademo.producer.VideoController : Video uploaded topic=demo-topic partition=0 offset=18 eventId=1bf0476f-8909-4791-9012-04a1b814d520 channelId=코딩애플 videoId=-fotMLYGM0c title=AI인데 글 생성을 못해서 난리난 Jev
```

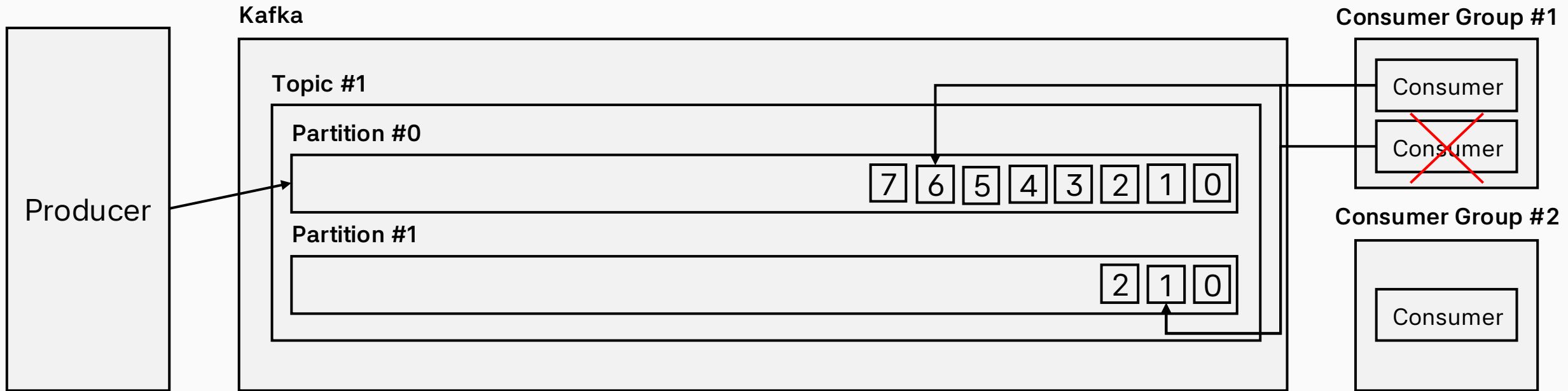
Consumer - analytics

```
T14:19:06.646+09:00 INFO 15768 --- [analytics-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Consumed video role=analytics topic=demo-topic partition=0 offset=18 eventId=1bf0476f-8909-4791-9012-04a1b814d520 channelId=코딩애플 videoId=-fotMLYGM0c
T14:19:06.646+09:00 INFO 15768 --- [analytics-consumer] [-listener-0-C-1] c.e.kafkademo.consumer.AnalyticsHandler : Video analytics eventId=1bf0476f-8909-4791-9012-04a1b814d520 channelId=코딩애플 videoId=-fotMLYGM0c channelUploads=1 totalUploads=3
T14:19:06.646+09:00 INFO 15768 --- [analytics-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Processing succeeded role=analytics topic=demo-topic partition=0 offset=18
T14:19:06.660+09:00 INFO 15768 --- [analytics-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Offset committed role=analytics topic=demo-topic partition=0 committedOffset=19
```

Consumer - notification 장애 후 복구

```
T14:19:49.022+09:00 INFO 21764 --- [notifications-consumer] [-listener-0-C-1] o.s.k.l.KafkaMessageListenerContainer : notifications-group: partitions assigned: [demo-topic-0]
T14:19:49.117+09:00 INFO 21764 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Consumed video role=notifications topic=demo-topic partition=0 offset=18 eventId=1bf0476f-8909-4791-9012-04a1b814d520 channelId=코딩애플 videoId=-fotMLYGM0c
T14:19:49.117+09:00 INFO 21764 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.NotificationHandler : Notification summary eventId=1bf0476f-8909-4791-9012-04a1b814d520 channelId=코딩애플 recipients=[] recipientCount=0
T14:19:49.117+09:00 INFO 21764 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Processing succeeded role=notifications topic=demo-topic partition=0 offset=18
T14:19:49.126+09:00 INFO 21764 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer : Offset committed role=notifications topic=demo-topic partition=0 committedOffset=19
```

Consumer 하나에 장애가 발생하면?



Consumer Group #1에서 어떤 Consumer에 장애가 발생하면 파티션 처리를 담당하는 Consumer를 재할당(리밸런스)
마찬가지로, 마지막으로 커밋된 오프셋부터 처리를 계속

3초마다 Broker 중 한 대에게 하트비트를 전송

45초의 세션 타임 아웃 내 하트비트가 수신됐는지 확인하고, 없으면 Consumer에 장애가 발생했다고 판단

Consumer 장애 시 리밸런스가 일어난다

GROUP	CURRENT-EPOCH	CONSUMER-ID	CURRENT-ASSIGNMENT	TARGET-EPOCH	TARGET-ASSIGNMENT	HOST	CLIENT-ID	#PARTITIONS
notifications-group	-	-	notification-2-0-a8e95ef4-f1b5-4536-adec-3f694a00a4c7	-	-	/10.89.0.4	notification-2-0 0	
notifications-group	-	-	notification-1-0-9c63275e-34c5-4c20-91f9-4d302a2d63be	-	-	/10.89.0.4	notification-1-0 1	
-	-	-	demo-topic:0	-	-	-	-	

Producer

```
2026-10-08T16:17:27.205+09:00 INFO 24304 --- [producer-app] [-app-producer-1] c.e.kafkademoproducer.VideoController
: Video uploaded topic=demo-topic partition=0 offset=26 eventId=dbfa7402-705a-4f6f-9202-be08bf0789b2 channelId=우아한테크
크 videoId=b65zIH7sDug title=회원시스템 이벤트기반 아키텍처 구축하기 #우아콘2022 #Day2_음식그 이상의것을문앞으로
```

Consumer - notification-1 장애 발생(16:17)

```
~\Documents\Projects\kafka > taskkill /PID 14040 /F
성공: 프로세스(PID 14040)가 종료되었습니다.
```

Consumer - analytics는 정상 수신

```
2026-10-08T16:17:27.205+09:00 INFO 19224 --- [analytics-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer
: Consumed video role=analytics topic=demo-topic partition=0 offset=26 eventId=dbfa7402-705a-4f6f-9202-be08bf0789b
2 channelId=우아한테크 videoId=b65zIH7sDug
2026-10-08T16:17:27.205+09:00 INFO 19224 --- [analytics-consumer] [-listener-0-C-1] c.e.kafkademconsumer.AnalyticsHan
dler : Video analytics eventId=dbfa7402-705a-4f6f-9202-be08bf0789b2 channelId=우아한테크 videoId=b65zIH7sDug channelUplo
ads=1 totalUploads=2
2026-10-08T16:17:27.205+09:00 INFO 19224 --- [analytics-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer
: Processing succeeded role=analytics topic=demo-topic partition=0 offset=26
2026-10-08T16:17:27.218+09:00 INFO 19224 --- [analytics-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer
: Offset committed role=analytics topic=demo-topic partition=0 committedOffset=27
```

Consumer 장애 시 리밸런스가 일어난다

Consumer - notification-2가 메시지 처리를 담당

notification-1 장애 후 리밸런스 발생

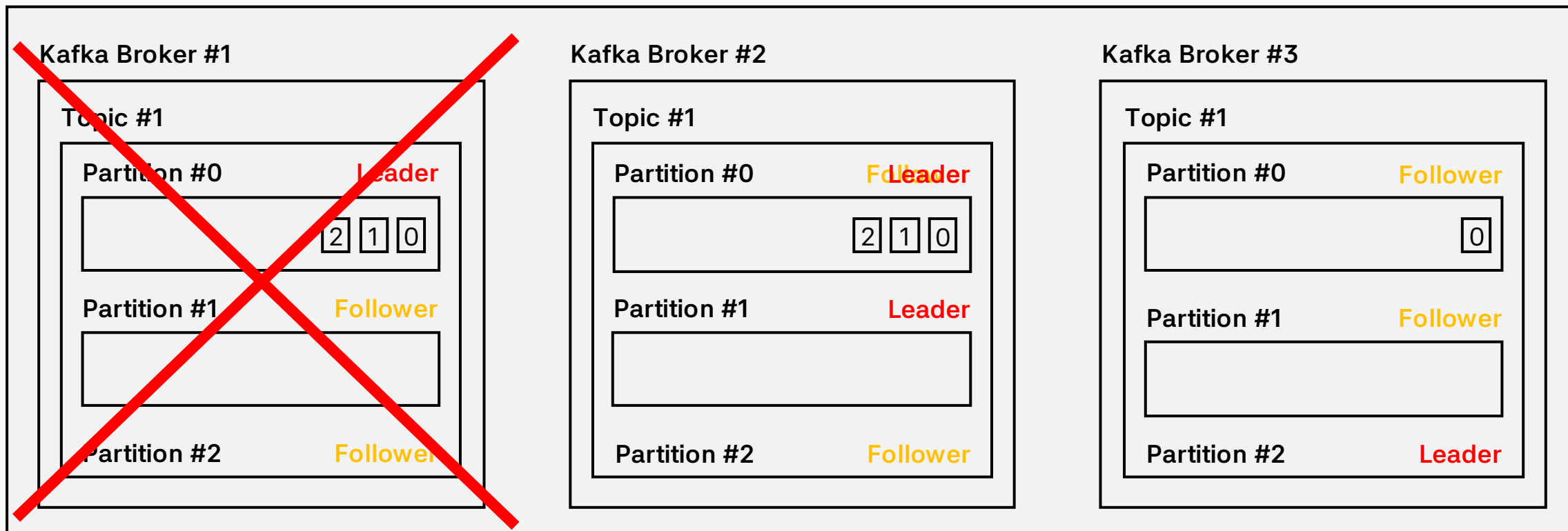
```
2026-10-08T16:18:09.806+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator: [Consumer clientId=notification-2-0, groupId=notifications-group] Request joining group due to: group is already rebalancing.
2026-10-08T16:18:09.806+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] k.c.c.i.ConsumerRebalanceListenerInvoker: [Consumer clientId=notification-2-0, groupId=notifications-group] Revoke previously assigned partitions
2026-10-08T16:18:09.813+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.s.k.l.KafkaMessageListenerContainer: notifications-group: partitions revoked: []
2026-10-08T16:18:09.813+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator: [Consumer clientId=notification-2-0, groupId=notifications-group] (Re-)joining group
2026-10-08T16:18:09.815+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator: [Consumer clientId=notification-2-0, groupId=notifications-group] Successfully joined group with generation Generation{generationId=18, memberId='notification-2-0-a8e95ef4-f1b5-4536-adec-3f694a00a4c7', protocol='range'}
2026-10-08T16:18:09.821+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator: [Consumer clientId=notification-2-0, groupId=notifications-group] Finished assignment for group at generation 18: {notification-2-0-a8e95ef4-f1b5-4536-adec-3f694a00a4c7=Assignment(partitions=[demo-topic-0])}
2026-10-08T16:18:09.823+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator: [Consumer clientId=notification-2-0, groupId=notifications-group] Successfully synced group in generation Generation{generationId=18, memberId='notification-2-0-a8e95ef4-f1b5-4536-adec-3f694a00a4c7', protocol='range'}
2026-10-08T16:18:09.823+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.a.k.c.c.internals.ConsumerCoordinator: [Consumer clientId=notification-2-0, groupId=notifications-group] Notifying assignor about the new Assignment(partitions=[demo-topic-0])
2026-10-08T16:18:09.823+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] k.c.c.i.ConsumerRebalanceListenerInvoker: [Consumer clientId=notification-2-0, groupId=notifications-group] Adding newly assigned partitions: demo-topic-0
2026-10-08T16:18:09.823+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.a.k.c.c.internals.ConsumerUtils: Setting offset for partition demo-topic-0 to the committed offset FetchPosition{offset=26, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[localhost:29092 (id: 1 rack: null)], epoch=11}}
2026-10-08T16:18:09.823+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] o.s.k.l.KafkaMessageListenerContainer: notifications-group partitions assigned: [demo-topic-0]
2026-10-08T16:18:09.924+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer: Consumed video role=notifications topic=demo-topic partition=0 offset=26 eventId=dbfa7402-705a-4f6f-9202-be08bf0789b2 channelId=우아한테크 videoId=b65zIH7sDug
2026-10-08T16:18:09.924+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.NotificationHandler: Notification summary eventId=dbfa7402-705a-4f6f-9202-be08bf0789b2 channelId=우아한테크 recipients=[] recipientCount=0
2026-10-08T16:18:09.924+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer: Processing succeeded role=notifications topic=demo-topic partition=0 offset=26
2026-10-08T16:18:09.924+09:00 INFO 8200 --- [notifications-consumer] [-listener-0-C-1] c.e.k.consumer.VideoEventConsumer: Offset committed role=notifications topic=demo-topic partition=0 committedOffset=27
```

GROUP	CONSUMER-ID	CURRENT-EPOCH	CURRENT-ASSIGNMENT	TARGET-EPOCH	TARGET-ASSIGNMENT	HOST	CLIENT-ID	#PARTITIONS
notifications-group	notification-2-0-a8e95ef4-f1b5-4536-adec-3f694a00a4c7	-	-	-	-	/10.89.0.4	notification-2-0 0	
notifications-group	notification-1-0-9c63275e-34c5-4c20-91f9-4d302a2d63be	-	-	-	-	/10.89.0.4	notification-1-0 1	
-	demo-topic:0	-	-	-	-	-	-	-

GROUP	CONSUMER-ID	CURRENT-EPOCH	CURRENT-ASSIGNMENT	TARGET-EPOCH	TARGET-ASSIGNMENT	HOST	CLIENT-ID	#PARTITIONS
notifications-group	notification-2-0-a8e95ef4-f1b5-4536-adec-3f694a00a4c7	-	-	-	-	/10.89.0.4	notification-2-0 1	
-	demo-topic:0	-	-	-	-	-	-	-

Broker 하나에 장애가 발생하면?

Kafka Cluster



Partition #0이 Leader인 Broker #1에 장애가 발생하는 경우 ISR인 Broker를 Leader로 선정

기본적으로 하트비트가 9초 동안 수신되지 않으면 Broker 장애라고 판단

ISR 선택 기준은 Replica의 배치 순서를 따름 (배치 순서는 고르게 분산되도록 StripedReplicaPlacer에 의해 결정됨)

Broker 장애가 발생하면 Leader 파티션이 변경된다

Broker가 모두 정상 상태일 때

```
14:47:01
Topic: demo-topic      TopicId: Pxi5g9QETA6GUszsPh630g PartitionCount: 1      ReplicationFactor: 3  C
onfigs: min.insync.replicas=2,segment.bytes=16777216,unclean.leader.election.enable=false
      Topic: demo-topic      Partition: 0      Leader: 1      Replicas: 1,2,3 Isr: 1,2,3      Elr: L
astKnownElr:
```

Broker-1 장애 발생

```
14:53:35
Topic: demo-topic      TopicId: Pxi5g9QETA6GUszsPh630g PartitionCount: 1      ReplicationFactor: 3  C
onfigs: min.insync.replicas=2,segment.bytes=16777216,unclean.leader.election.enable=false
      Topic: demo-topic      Partition: 0      Leader: 2      Replicas: 1,2,3 Isr: 2,3      Elr: L
astKnownElr:
```

Broker-1 장애 복구 후

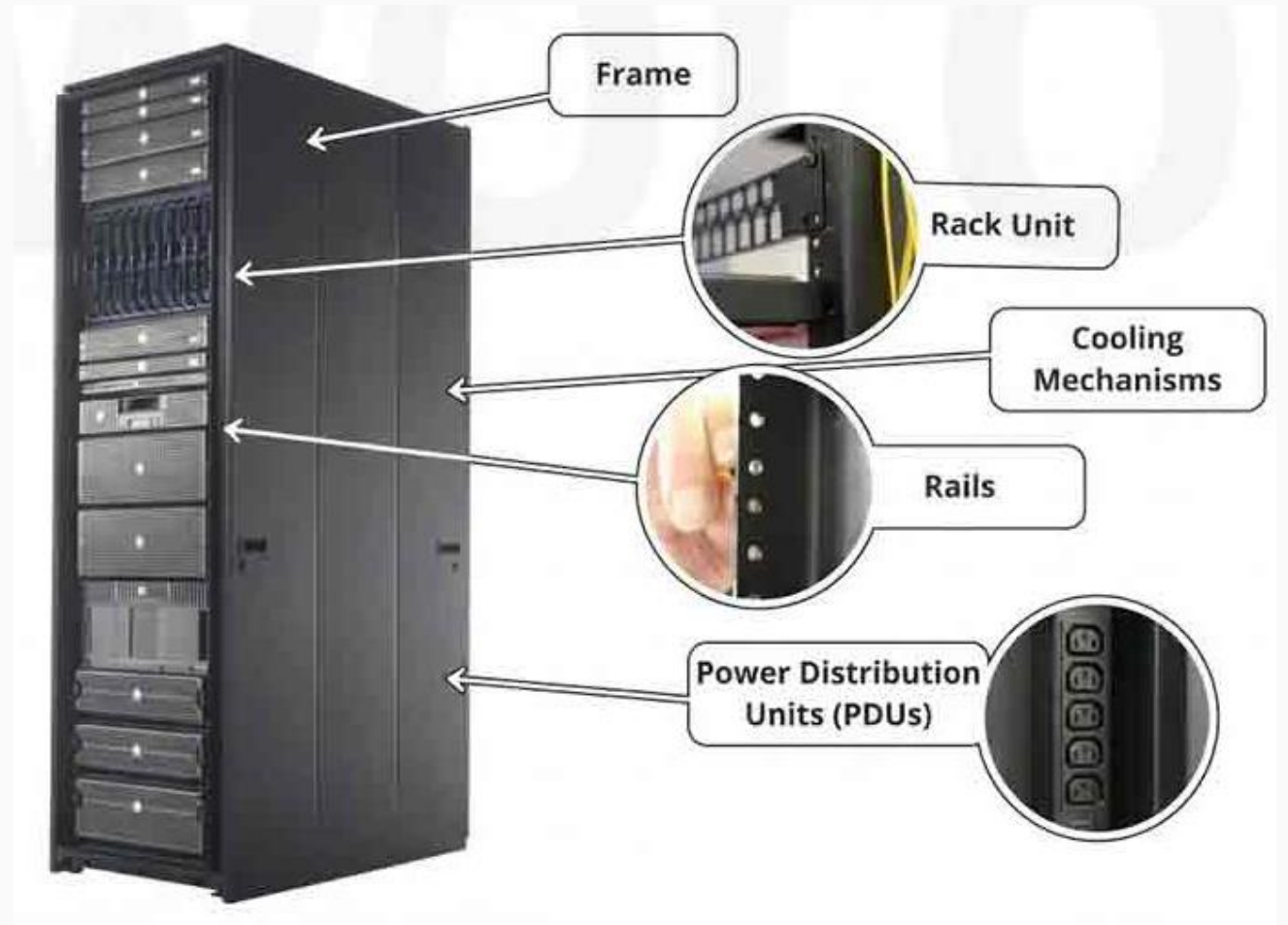
```
14:54:31
Topic: demo-topic      TopicId: Pxi5g9QETA6GUszsPh630g PartitionCount: 1      ReplicationFactor: 3  C
onfigs: min.insync.replicas=2,segment.bytes=16777216,unclean.leader.election.enable=false
      Topic: demo-topic      Partition: 0      Leader: 2      Replicas: 1,2,3 Isr: 2,3      Elr: L
astKnownElr:
14:54:35
Topic: demo-topic      TopicId: Pxi5g9QETA6GUszsPh630g PartitionCount: 1      ReplicationFactor: 3  C
onfigs: min.insync.replicas=2,segment.bytes=16777216,unclean.leader.election.enable=false
      Topic: demo-topic      Partition: 0      Leader: 2      Replicas: 1,2,3 Isr: 1,2,3      Elr: L
astKnownElr:
```

Broker 하나에 장애가 발생하면?

서버 랙에 Broker를 물리게 구성 시

- PDU 장애
- 네트워크 스위치 장애
- 전원 회로 장애
- 물리적 장비 장애

같은 Partition의 Replica를
서로 다른 랙의 Broker에 분산해야 함



Wolontek

RabbitMQ

많은 기업에서 사용하고 있는 Kafka

Kafka에서 사용하는 용어와 매커니즘

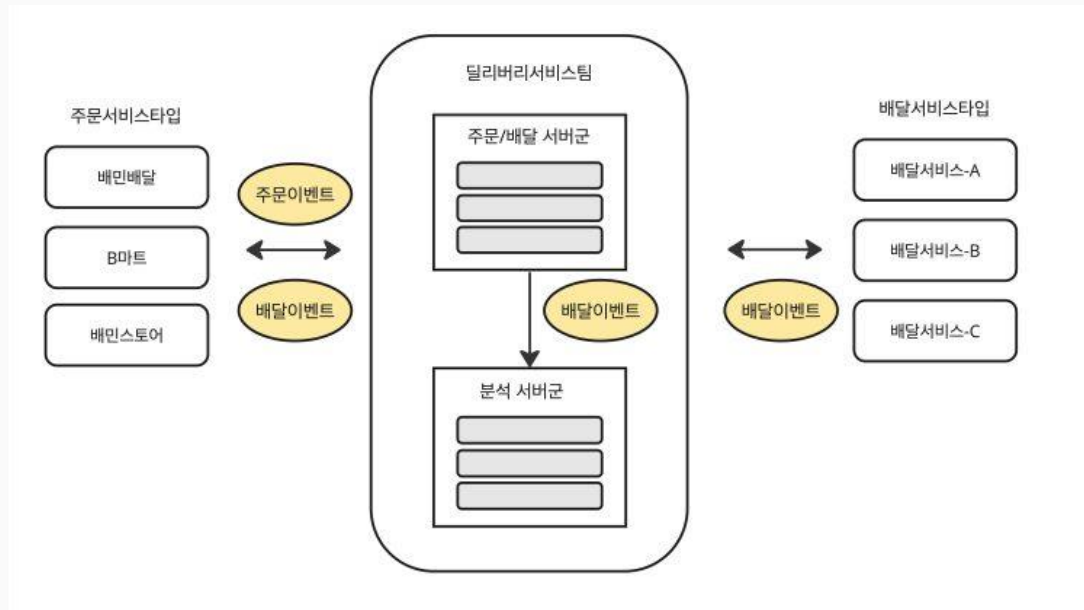
Kafka, 어떻게 활용하고 있을까?

배달의민족

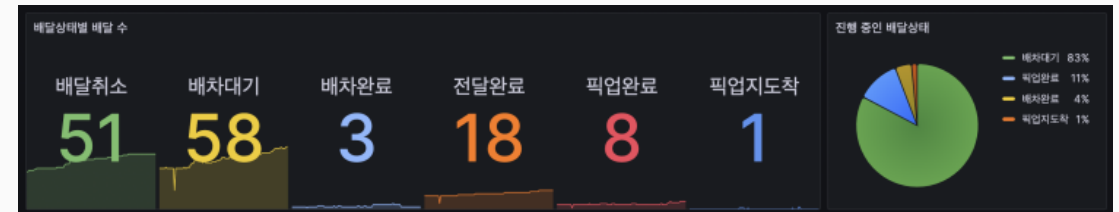
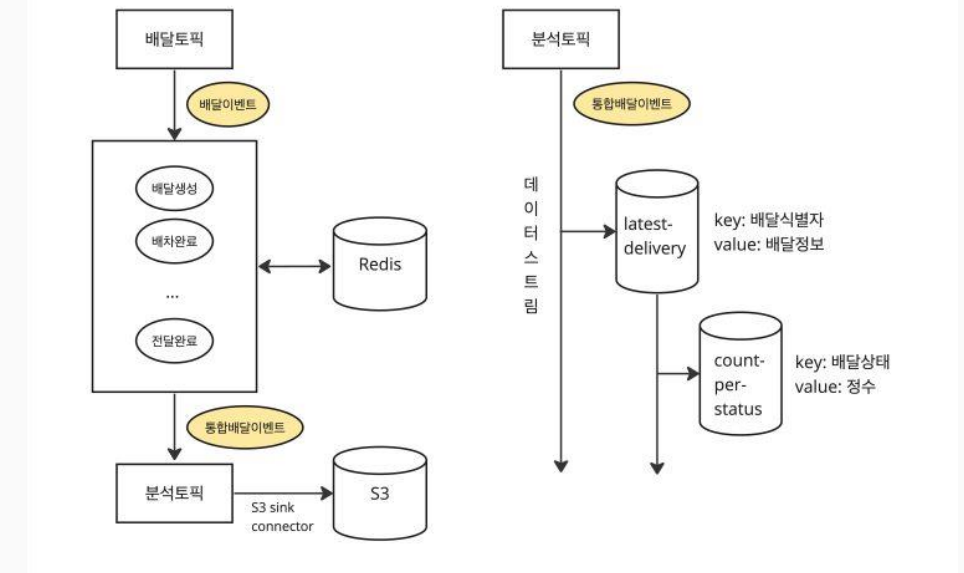
카프카 클러스터 규모 (2022년 8월 운영 환경 기준)

- 카프카 클러스터 27개
- 브로커 162대
- 파티션 약 24,000개
- 초당 평균 50만건, 최대 140만건 메시지 유입

주문과 배달 처리

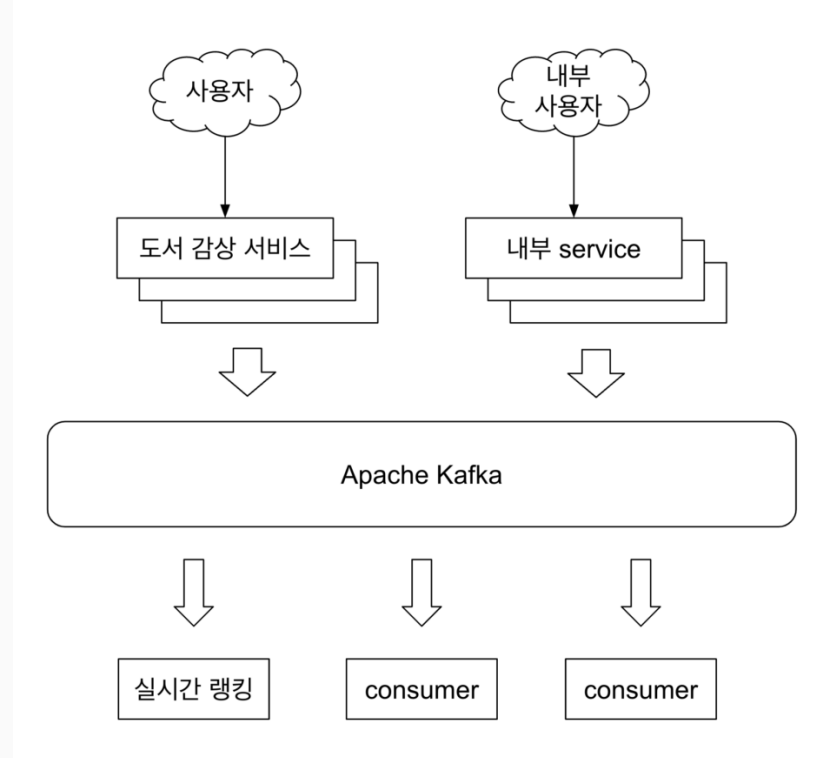


준실시간 배달 현황 분석

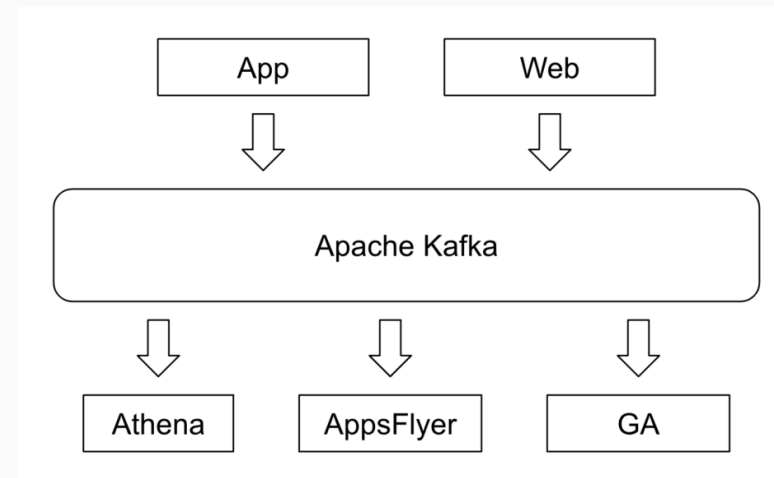


RIDI

실시간 랭킹



애널리틱스



실시간 부정행위 감시 및 실시간 보상 지급

**웹툰 보면
전원 포인트!**

웹툰을 유료로 감상한
화수에 따라 즉시 포인트를 드립니다.

CJ올리브영

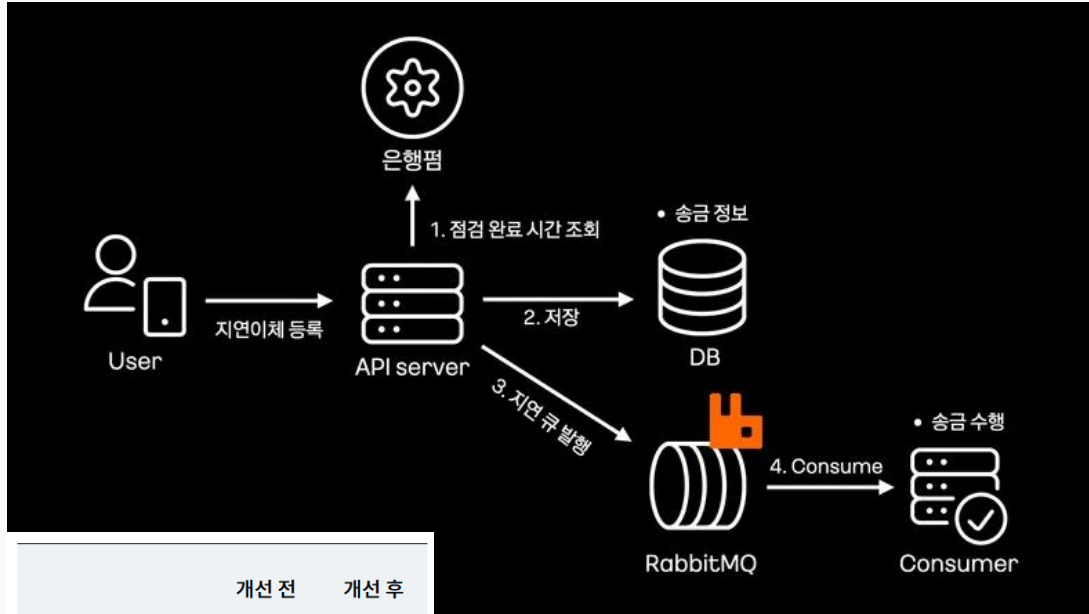


★ 전시혜택정보테이블 모니터링(PRD) >

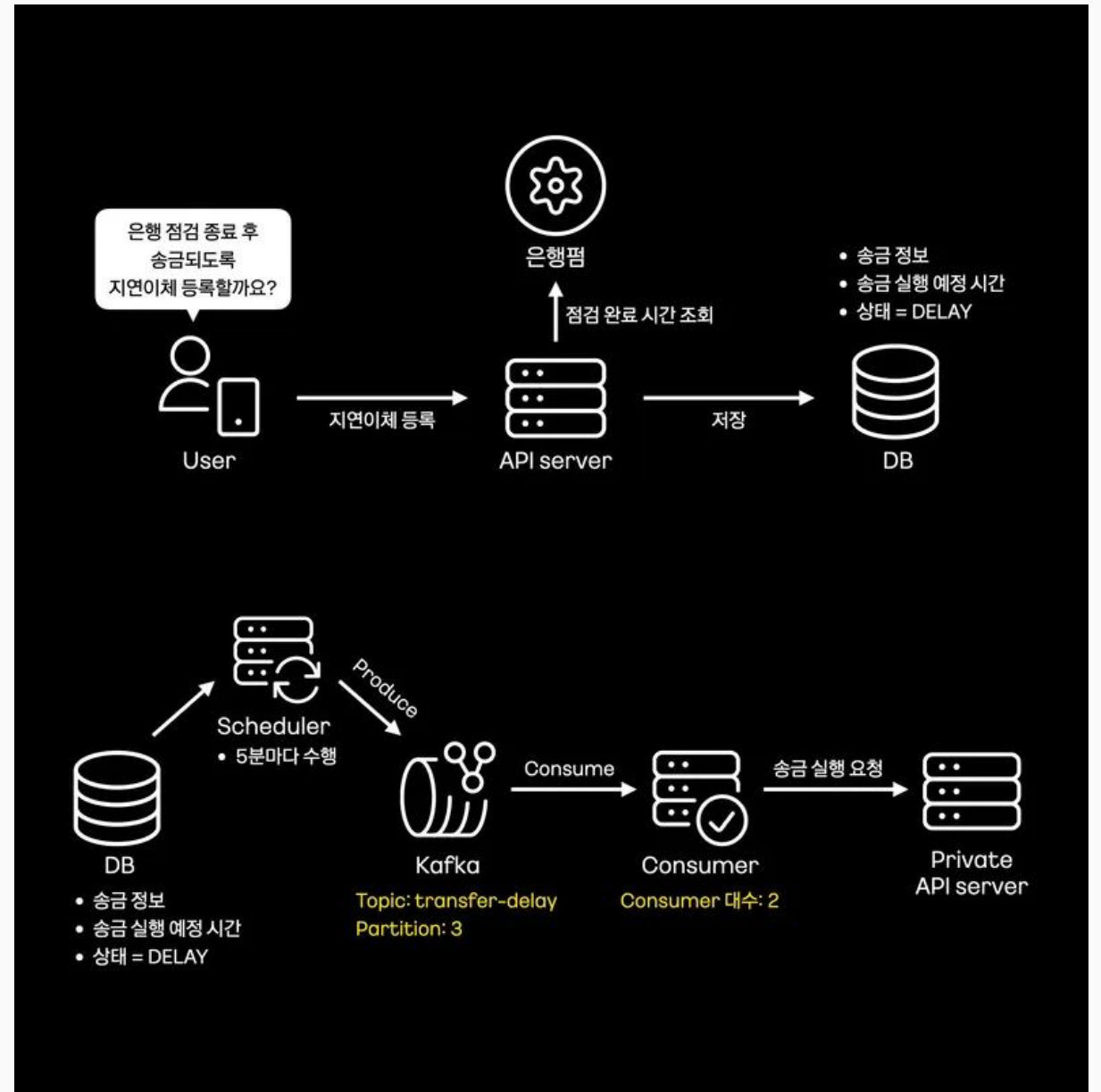
Share Show Overlay Configure Add Widgets

상품	가격	증정프로모션
100% 100%	58 100%	3 100%

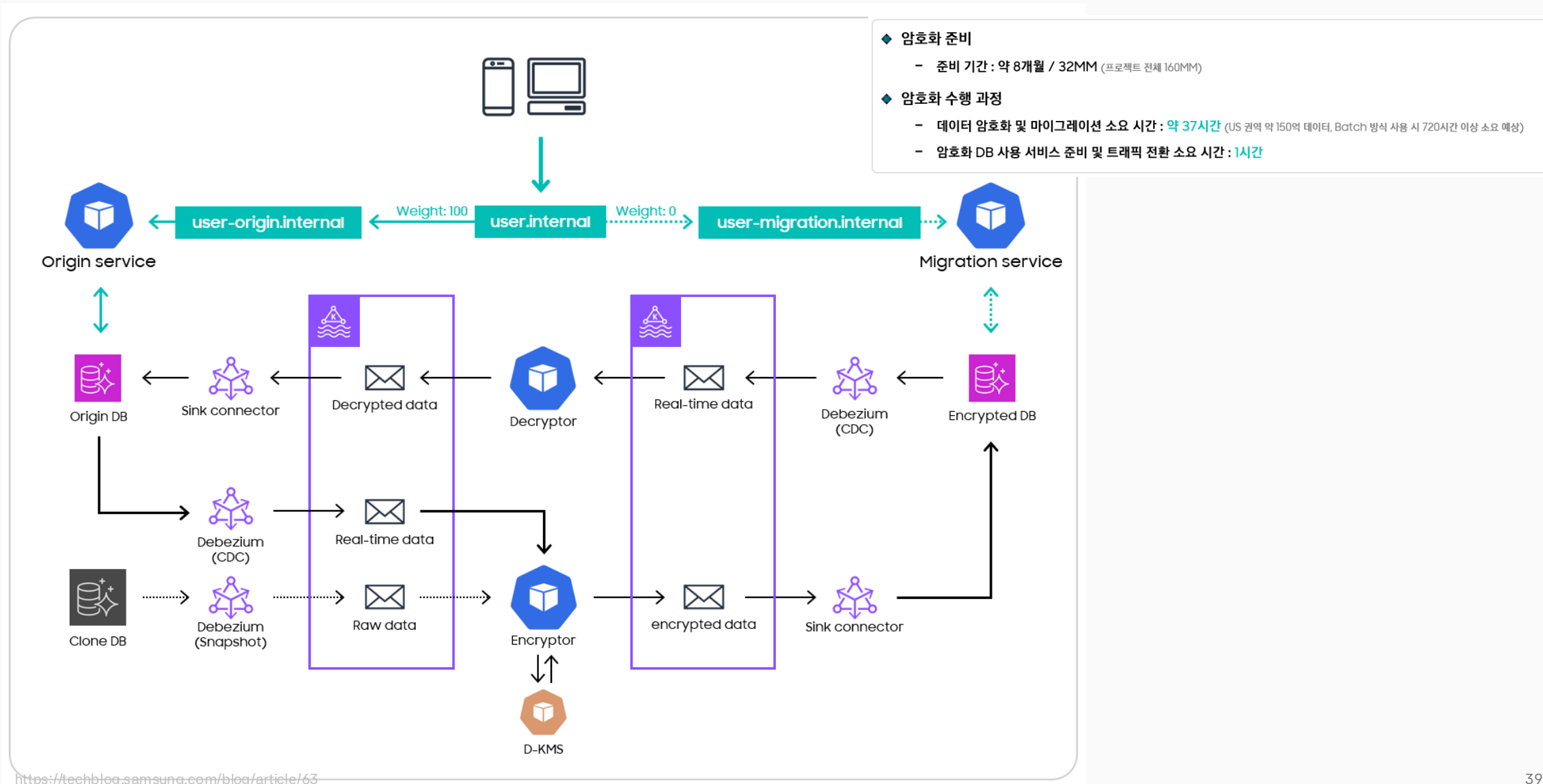
카카오페이



	개선 전	개선 후
Consumer 대수	2	3
Read Records 수	1	20
Thread 수	1	10
1분 당 처리 건수	91건	728건
실행 시간	68분	8분



삼성 계정 무중단 컬럼 단위 암호화 및 마이그레이션



RabbitMQ와 Kafka

RabbitMQ

사용 목적	메시지 전달, 애플리케이션 간 통신, 백그라운드 작업 처리
아키텍처	메시지 브로커
메시지 저장	메모리 기반
처리량	초당 수백만 개의 메시지
확장성	수평 확장 가능하나 복잡
라우팅	다양하고 복잡한 라우팅 지원
메시지 보존	소비 후 일반적으로 삭제
적합 사례	알림 발송, 이미지 변환, 백그라운드 작업 등 특정 Consumer에게 전달해 처리하는 작업 중심 구조

Apache Kafka

대용량 데이터 저장 및 스트리밍, 실시간 데이터 파이프 라인 및 스트림 처리 분산 스트리밍 플랫폼
디스크 기반
초당 수십억 개의 메시지
수평 확장이 쉽고 효율적
단순한 토픽 기반 구조
설정 기간 동안 보존 후 삭제 또는 압축
메시지를 여러 서비스에서 사용하는 이벤트 스트리밍 구조

Observer 패턴과 Pub/Sub 패턴

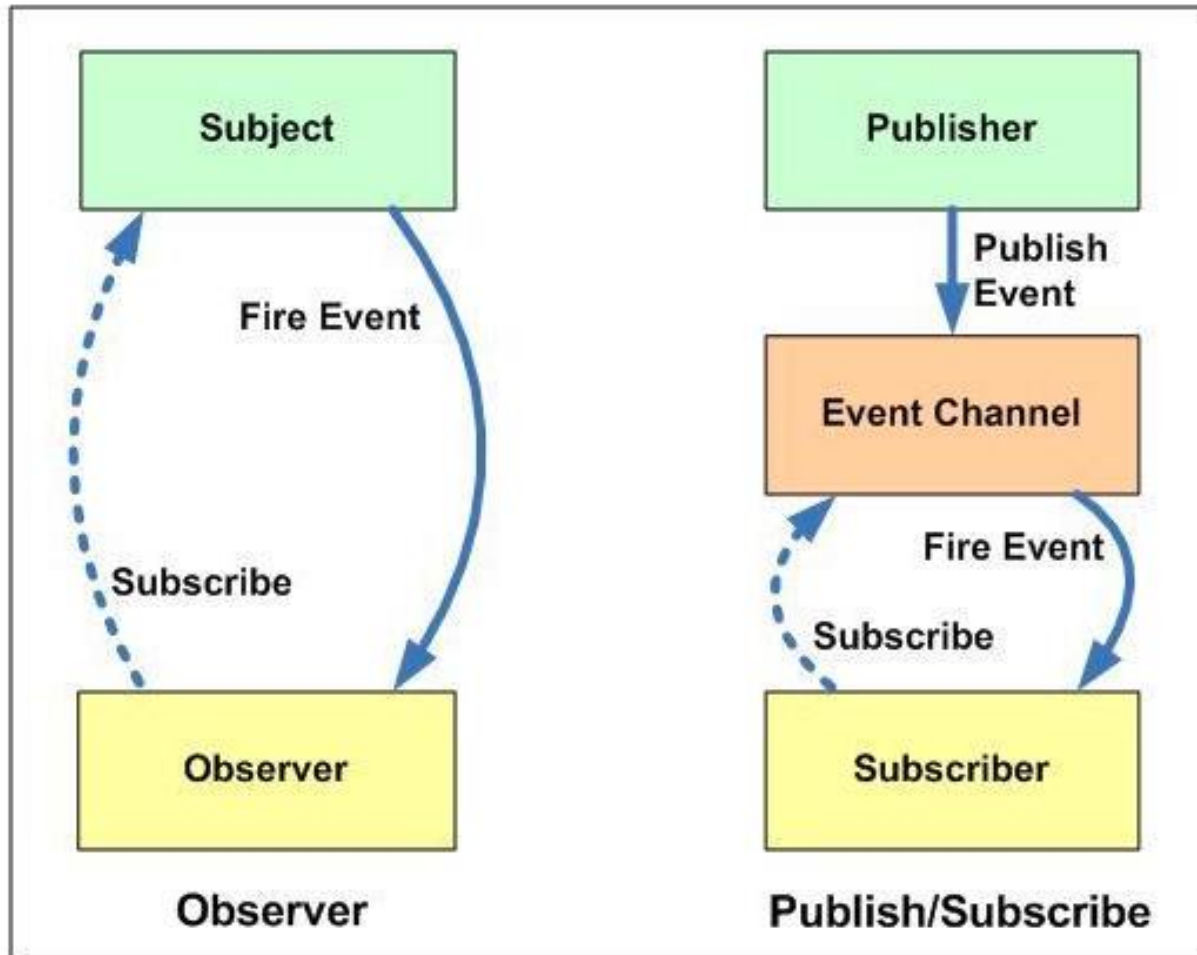


Image source: [developers-club](#)

Observer 패턴은 Subject와 Observer가 강하게 연결
Pub/Sub 패턴은 Broker 덕에 느슨하게 연결

Observer 패턴은 단일 도메인에서 처리되어야 함
Pub/Sub 패턴은 Broker 덕에 언어 등
도메인이 달라도 처리 가능

들어 주셔서 감사합니다

박연종